Artificial Intelligence - Report Lab 6 & 7

Anh Khoi Pham - ITCSIU23018

May 21, 2026

1 Problem 1

Implement the `fit()` method, which takes in arguments `X` (the training data) and `y` (the training labels) to count the number of occurrences of each word and record all information in a DataFrame.

```
def fit(self, X, y):
    self.vocabulary = set(word for sms in X for word in sms)
    self.parameters_ham = {word: 0 for word in self.vocabulary}
    self.parameters_spam = {word: 0 for word in self.vocabulary}

    rows = []
    sms_spam = 0
    sms_ham = 0

    for i, sms in enumerate(X):
        row = {word: 0 for word in self.vocabulary}

        sms_spam += 1 if y.iloc[i] == 'spam' else 0
        sms_ham += 1 if y.iloc[i] == 'ham' else 0

        for word in sms:
            row[word] += 1
            if y.iloc[i] == 'spam':
                self.parameters_spam[word] += 1
                self.n_words_spam += 1
            else:
                self.parameters_ham[word] += 1
                self.n_words_ham += 1

        row['Label'] = y.iloc[i]
        rows.append(row)

    self.data = pd.DataFrame(rows)
    self.p_spam = sms_spam / len(X)
    self.p_ham = sms_ham / len(X)

    return self.data
```

Here is how the code works:

1. For each message in `X`, create a bag-of-words representation using the set of words that appear in the input data.
2. Update the bag-of-words while iterating through each word in a message.
3. Save it as a DataFrame and return.

```
NB = NaiveBayesFilter()
NB.fit(X_train, y_train)
```

20.6s Python

```
parameters_spam:  {'opt-out': 4, 'weed"': 0, 'cds': 4, 'wrk.i': 0, '(…)': 0, 'topic': 0, 'left.': 0, 'lesson...': 0, 'e-namous.': 0, 'staff': 0,
parameters_ham:  {'opt-out': 0, 'weed"': 1, 'cds': 0, 'wrk.i': 1, '(…)': 1, 'topic': 1, 'left.': 2, 'lesson...': 2, 'e-namous.': 1, 'staff': 2,
```

	opt-out	weed"	cds	wrk.i	(…)	topic	left.	lesson...	e-namous.	staff	…	bathing...	caroline	delicious	sport,	messages-text	dreams..	loves	80122300p/wk	corru
0	0	0	0	0	0	0	0	0	0	0	…	0	0	0	0	0	0	0	0	
1	0	0	0	0	0	0	0	0	0	0	…	0	0	0	0	0	0	0	0	
2	0	0	0	0	0	0	0	0	0	0	…	0	0	0	0	0	0	0	0	
3	0	0	0	0	0	0	0	0	0	0	…	0	0	0	0	0	0	0	0	
4	0	0	0	0	0	0	0	0	0	0	…	0	0	0	0	0	0	0	0	
…	…	…	…	…	…	…	…	…	…	…	…	…	…	…	…	…	…	…	…	
4453	0	0	0	0	0	0	0	0	0	0	…	0	0	0	0	0	0	0	0	
4454	0	0	0	0	0	0	0	0	0	0	…	0	0	0	0	0	0	0	0	
4455	0	0	0	0	0	0	0	0	0	0	…	0	0	0	0	0	0	0	0	
4456	0	0	0	0	0	0	0	0	0	0	…	0	0	0	0	0	0	0	0	
4457	0	0	0	0	0	0	0	0	0	0	…	0	0	0	0	0	0	0	0	

4458 rows × 11861 columns

2 Problem 2

Calculate the necessary probabilities for implementing the Naive Bayes Classification.

```
def predict_proba(self, X):
    proba = []

    for sms in X:
        p_mul_spam = self.p_spam
        p_mul_ham = self.p_ham

        for word in sms:
            # P(Xi|spam) and P(Xi|ham)
            p_xi_spam = (self.parameters_spam.get
               (word, 0) + 1) / (self.
               n_words_spam + len(self.vocabulary
               ))
```

```
11                p_xi_ham = (self.parameters_ham.get(
                      word, 0) + 1) / (self.n_words_ham
                      + len(self.vocabulary))
12
13                # P(spam|Xi) and P(ham|Xi)
14                p_mul_spam *= p_xi_spam
15                p_mul_ham *= p_xi_ham
16
17            proba.append({'P(spam|X)': p_mul_spam, 'P
                  (ham|X)': p_mul_ham})
18
19        return proba
```

In detail, this code calculates the probability of a message being spam or ham using the Naive Bayes formula. For a message X with words $x_1, x_2, \ldots, x_n$, the probabilities are computed as:

$$P(\text{spam} \mid X) \propto P(\text{spam}) \prod_{i=1}^{n} P(x_i \mid \text{spam})$$

$$P(\text{ham} \mid X) \propto P(\text{ham}) \prod_{i=1}^{n} P(x_i \mid \text{ham})$$

The conditional probabilities are calculated with Laplace smoothing as follows:

$$P(x_i \mid \text{spam}) = \frac{N_{x_i,\text{spam}} + 1}{N_{\text{spam}} + |V|}$$

$$P(x_i \mid \text{ham}) = \frac{N_{x_i,\text{ham}} + 1}{N_{\text{ham}} + |V|}$$

where $N_{x_i,\text{spam}}$ and $N_{x_i,\text{ham}}$ are the number of times word x_i appears in spam and ham messages, N_{spam} and N_{ham} are the total number of words in spam and ham messages, and $|V|$ is the vocabulary size.

3 Problem 3

Complete the the `predict()` method, return "spam" or "ham" for each message based on the probabilities calculated from Problem 2.

```
1 def predict(self, X):
2     prob = self.predict_proba(X)
```

```
    predict_labels = []

    for p in prob:
        if p['P(spam|X)'] > p['P(ham|X)']:
            predict_labels.append('spam')
        else:
            predict_labels.append('ham')

    return predict_labels
```

```
predict_labels = NB.predict(X_test)
print("predict_labels: ", predict_labels[:5])
[15] ✓ 0.0s                                                    Python
predict_labels:  ['ham', 'ham', 'spam', 'ham', 'ham']
```

4 Problem 4

Calculate the recall to evaluate our model.

Recall measures how many actual spam messages are correctly classified as spam. It is calculated as:

$$\text{Recall} = \frac{TP}{TP + FN}$$

where TP is the number of true positives, meaning spam messages predicted as spam, and FN is the number of false negatives, meaning spam messages predicted as ham.

```
def score(self, true_labels, predict_labels):
    recall = 0
    tp = sum(1 for true, pred in zip(true_labels,
        predict_labels) if true == 'spam' and
       pred == 'spam')
    fn = sum(1 for true, pred in zip(true_labels,
        predict_labels) if true == 'spam' and
       pred == 'ham')
    if tp + fn > 0:
        recall = tp / (tp + fn)

    return recall
```

```
    recall = NB.score(y_test, predict_labels)
    print("recall of NB: ", recall)
[14]  ✓ 0.0s                                                                    Python
...  recall of NB:  0.9455782312925171
```

5 Results

```
(ai-lab) pham-anh-khoi@pham-anh-khoi-ASUS-TUF-Gaming-F15-FX507ZU4-FX507ZU4:~/Study/SEM2_2025-2026/AI/Artificial-Intelligent-Lab/PhamAnhKhoi_ITCS
IU23018_Lab6&7$ python spam_filter.py
(5572, 2)
------------------BEFORE CLEANING DATA-------------------------
  Label                                                SMS
0   ham  Go until jurong point, crazy.. Available only ...
1   ham                      Ok lar... Joking wif u oni...
2  spam  Free entry in 2 a wkly comp to win FA Cup fina...
3   ham  U dun say so early hor... U c already then say...
4   ham  Nah I don't think he goes to usf, he lives aro...
Total message:  5572
training_test_index:  4458
Length of test data:  1114
----AFTER CLEANING AND SPLIT TRAIN AND TEST DATA----
0                  [yep,, by, the, pretty, sculpture]
1    [yes,, princess., are, you, going, to, make, m...
2                    [welp, apparently, he, retired]
3                                           [havent.]
4    [i, forgot, 2, ask, ü, all, smth.., there's, a...
Name: SMS, dtype: object
recall train of NB:  0.9633333333333334
recall test of NB:  0.9455782312925171
```